

CTR Prediction for Mobile Advertisement Personalisation

Candidate 65013	Contribution: 20%
Candidate 68923	Contribution: 20%
Candidate 68189	Contribution: 20%
Candidate 63593	Contribution: 20%
Candidate 62665	Contribution: 20%

1. Introduction

Click-Through-Rate (CTR) prediction is a core aspect of personalised web-advertisement systems. Given contextual data about the user and ad-related features, the objective is to estimate the probability of a user clicking on a displayed advertisement. Accurate CTR models are therefore commercially relevant, driving higher revenue and improved customer experiences.

However, developing effective models faces many challenges. Firstly, click events are rare in comparison to impressions, leading to strongly imbalanced training data. Secondly, some inputs are high-cardinality features, which are typically represented using one-hot encoding, leading to exacerbated sparsity of data. Finally, the modelling of feature interactions heavily influences the predictive power of the models. To illustrate, middle-aged men are more inclined to click on health related advertisements than a teenage boy, suggesting that a second-order interaction between age and the category of the advertisement is pivotal in prediction. An effective CTR model must therefore handle highly imbalanced datasets and capture feature interactions.

Google’s Wide&Deep (Cheng et al., 2016) demonstrated that combining the outputs of both low and high-order feature interaction models (Logistic Regression and Multilayer Perceptron (MLP) respectively) achieves improved performance in their experiments. DeepFM (Guo et al., 2017) builds upon this by combining Factorisation Machines (FM), built to capture second-order feature interactions via latent vectors inner products, with MLP models that capture high-order interactions. It improves upon Wide&Deep, as both the FM and MLP components share the same input and embedding matrix, reducing the need for manual feature engineering. In relation to performance, DeepFM has shown greater performance than most deep learning and classification models, including Wide&Deep, Logistic Regression, CNNs and RNNs (Yang and Zhai, 2022).

However, the FM component of DeepFM assigns all pairwise interactions equal weight-

ing, despite interaction importance varies by feature pair. For example, the interaction of a user’s age and content category is likely to have higher weighting than type of content and time of the day. This has led to the development of models that combine the MLP component with other improved FM models (Zhang et al., 2019), such as Field-aware Factorisation machines (FFM) (Juan et al., 2017). FFM learns field-specific interactions, meaning the embedding of feature i depends on the field of the feature it interacts with. This leads to a high amount of factors (approximately quadratic in the number of features) in the model, which is much heavier in memory and training time, especially with high-cardinality sparse data. As an alternative to FFM, Attentional Factorisation machines (AFM) (Xiao et al., 2017) have a better runtime, where a neural attention network is used to understand the importance of the interactions between features in the data. Motivated by this, we proposed a way to integrate AFM into the current structure of DeepFM to produce a model that effectively captures both low and high order feature interactions while reducing the impact of uninformative interactions in the second-order component.

Our main contributions are summarized below:

- We propose an adaptation of DeepFM, **AFM-MLP** that integrates AFM’s attention-weighted feature interactions with an MLP branch to capture both low-order and high-order feature interactions without manual feature engineering.
- Unlike DeepFM which treats all feature interactions equally, our attention mechanism assigns learnable importance weights to pairwise interactions, reducing noise while maintaining computational efficiency compared to FFM-based approaches. We also experimented with the AdamW optimiser rather than Adam to enable decoupled weight decay and to study the effect of explicit regularisation on the learned embedding and attention parameters.
- We provide a controlled empirical comparison between Logistic Regression, FM, AFM, DeepFM, and AFM-MLP on Avazu using AUC and log-loss

2. Problem Formulation

2.1 Notation Definition

We consider a dataset $\mathcal{A} = \{(\chi_i, y_i)\}_{i=1}^N$, where χ_i consists of m categorical fields and $y_i \in \{0, 1\}$ indicates whether the impression resulted in a click. Each instance is represented by a sparse one-hot vector $\mathbf{x} \in \{0, 1\}^n$, with exactly one active category per field.

Embedding: Let k denote the embedding dimension, $\mathbf{V} \in \mathbb{R}^{n \times k}$ the full embedding matrix where each row corresponds to a feature, and $\mathbf{V}_j \in \mathbb{R}^{n_j \times k}$ the sub-matrix of $\mathbf{V}$ corresponding to field j . Let $\mathbf{v}_i \in \mathbb{R}^k$ denote the embedding (latent) vector for feature i . For field j , the field embedding is $\mathbf{v}_j = \mathbf{V}_j^\top \mathbf{x}_{\text{field}_j} \in \mathbb{R}^k$. Since each field is one-hot encoded, this retrieves the embedding vector of the active feature in field j . We denote $\mathcal{E} = \{\mathbf{v}_1, \dots, \mathbf{v}_m\}$ as the set of all field embeddings.

AFM. Let $\mathcal{X}$ denote the set of indices of non-zero features in $\mathbf{x}$. Define the set of unordered feature pairs as

$$\mathcal{R}_{\mathbf{x}} = \{(i, j) : i, j \in \mathcal{X}, i < j\}, \quad |\mathcal{R}_{\mathbf{x}}| = \binom{m}{2}.$$

Let $x_i \in \{0, 1\}$ denote the value of feature i . Since $\mathbf{x}$ is one-hot encoded per field, we have $x_i = 1$ for all $i \in \mathcal{X}$, and hence $x_i x_j = 1$ for all $(i, j) \in \mathcal{R}_{\mathbf{x}}$.

Basic operations. For embeddings $\mathbf{v}_i, \mathbf{v}_j \in \mathbb{R}^k$, define the inner product

$$\langle \mathbf{v}_i, \mathbf{v}_j \rangle = \sum_{\ell=1}^k v_{i\ell} v_{j\ell},$$

and the Hadamard product $\odot$ by

$$(\mathbf{v}_i \odot \mathbf{v}_j)_{\ell} = v_{i\ell} \cdot v_{j\ell}, \quad \mathbf{v}_i \odot \mathbf{v}_j \in \mathbb{R}^k.$$

FM terms. Let $w_0 \in \mathbb{R}$ denote the global bias, $w_i \in \mathbb{R}$ the first-order weight for feature i , and let the factorised interaction weight be

$$\hat{w}_{ij} = \langle \mathbf{v}_i, \mathbf{v}_j \rangle \in \mathbb{R}.$$

Pairwise interaction layer. Define the pairwise interaction layer as

$$f_{PI}(\mathcal{E}) = \{(\mathbf{v}_i \odot \mathbf{v}_j) x_i x_j\}_{(i,j) \in \mathcal{R}_{\mathbf{x}}},$$

which produces $\binom{m}{2}$ interaction vectors, each in $\mathbb{R}^k$.

Attention network and pooling. Let t be the attention factor (hidden dimension), with parameters $\mathbf{W} \in \mathbb{R}^{t \times k}$, $\mathbf{b} \in \mathbb{R}^t$, and context vector $\mathbf{h} \in \mathbb{R}^t$. Let $\text{ReLU}(z) = \max(0, z)$. For each pair $(i, j) \in \mathcal{R}_{\mathbf{x}}$, define the unnormalised attention score

$$a'_{ij} = \mathbf{h}^{\top} \text{ReLU}(\mathbf{W}(\mathbf{v}_i \odot \mathbf{v}_j) + \mathbf{b}) \in \mathbb{R},$$

and the normalised attention weight

$$a_{ij} = \frac{\exp(a'_{ij})}{\sum_{(l,r) \in \mathcal{R}_{\mathbf{x}}} \exp(a'_{lr})} \in (0, 1), \quad \sum_{(i,j) \in \mathcal{R}_{\mathbf{x}}} a_{ij} = 1.$$

The attention-based pooled representation is then

$$f_{Att}(f_{PI}(\mathcal{E})) = \sum_{(i,j) \in \mathcal{R}_{\mathbf{x}}} a_{ij} (\mathbf{v}_i \odot \mathbf{v}_j) x_i x_j \in \mathbb{R}^k,$$

and $\mathbf{p} \in \mathbb{R}^k$ denotes the projection vector.

MLP: Let H denote the number of hidden layers and $\ell \in \{1, \dots, H\}$ the layer index. Let $\mathbf{a}^{(0)} = [\mathbf{v}_1, \dots, \mathbf{v}_m] \in \mathbb{R}^{d_0}$ denote the MLP input formed by concatenating all field embeddings, where $d_0 = k \times m$ is the input dimension. For layer ℓ , let $\mathbf{W}^{(\ell)} \in \mathbb{R}^{d_{\ell} \times d_{\ell-1}}$ denote the weight matrix, $\mathbf{b}^{(\ell)} \in \mathbb{R}^{d_{\ell}}$ the bias vector, and d_{ℓ} the number of neurons. Let $\sigma(\cdot)$ denote an element-wise activation function, and $\mathbf{a}^{(\ell)} = \sigma(\mathbf{W}^{(\ell)} \mathbf{a}^{(\ell-1)} + \mathbf{b}^{(\ell)}) \in \mathbb{R}^{d_{\ell}}$ the output of layer ℓ . For the output layer, let $\mathbf{W}^{(H+1)} \in \mathbb{R}^{1 \times d_H}$ and $b^{(H+1)} \in \mathbb{R}$ denote the projection weights and bias, with $y_{\text{MLP}} = \mathbf{W}^{(H+1)} \mathbf{a}^{(H)} + b^{(H+1)} \in \mathbb{R}$ the MLP output.

2.2 Input and Output Space

Suppose the dataset for training consists of N instances (χ, y) , where χ is an m -field data record representing user and the advertisement information, and $y \in \{0, 1\}$ is the associated label indicating user click behaviour ($y = 1$ the user clicked the ad, and $y = 0$ otherwise).

Each field in χ corresponds to a categorical feature such as user information, ads type or hour of day. Each categorical field is represented as a vector via one-hot encoding. Then, each instance is converted to $(\mathbf{x}, y)$ where:

$$\mathbf{x} = [\mathbf{x}_{\text{field}_1}, \mathbf{x}_{\text{field}_2}, \dots, \mathbf{x}_{\text{field}_m}] \quad (1)$$

is an n -dimensional vector, with $\mathbf{x}_{\text{field}_j}$ being the one-hot encoded representation of the j -th field. The total dimension is $n = \sum_{j=1}^m n_j$, where n_j is the number of unique categories in field j . Therefore, this can lead to a highly sparse datapoint, a fundamental problem in CTR prediction.

The output $y \in \{0, 1\}$ is a binary label where $y = 1$ indicates a click on the advertisement and $y = 0$ indicating no click. The model outputs a predicted probability $\hat{y} \in (0, 1)$ representing the estimated click-through rate:

$$\hat{y} = P(y = 1 \mid \mathbf{x}) \quad (2)$$

2.3 Loss Function

We minimise the Log-Loss function to train our model, given by:

$$J(\boldsymbol{\theta}) = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] \quad (3)$$

where $\boldsymbol{\theta}$ is the model parameter vector.

Each predicted probability may be interpreted as the parameter of a Bernoulli distribution. Under our observation that each outcome is independent and identically distributed, we find that the likelihood of the training set factorises into the product of individual Bernoulli likelihoods. Training the model is therefore equivalent to maximising the likelihood of the observed data, which is equivalent to minimising the Log-Loss function.

2.4 Optimisation

This section is written in reference to Loshchilov and Hutter (2019). We train our model using the AdamW optimiser, over Adam with L2 regularisation. Adam with L2 is suboptimal as the regularisation term is scaled by the adaptive learning rate, which causes inconsistent weight decay across our parameters. AdamW decouples weight decay from the gradient update, applying it directly to the parameters instead of the loss function. This leads to more stable optimisation as the regularisation is applied independently of Adam’s adaptive learning rates.

We provide a sketch of the algorithm below. The following description follows the structure of Algorithm 2 in Loshchilov and Hutter (2019). We begin by initialising: $\boldsymbol{\theta}_0$ as the starting parameter vector, $\mathbf{m}_t$ as the first moment vector, $\mathbf{v}_t$ as the second moment vector, $\alpha > 0$ as the learning rate, $\lambda \geq 0$ as the weight decay, $\beta_1, \beta_2 \in (0, 1)$ as the decay rates,

$\epsilon > 0$ as a constant, $\gamma_{t=0} \in \mathbb{R}$ as the schedule multiplier, and $t = 0$ as our starting time. We note that the $\mathbf{m}_t$ and $\mathbf{v}_t$ have the same dimension as the parameter vector $\boldsymbol{\theta}_t$ and are both initialised to $\mathbf{0}$.

Then for $t = 1, 2, \dots$ until a stopping criterion is met, we iteratively compute the following steps:

1. Randomly select a sample $\mathcal{S}_t \subset \mathcal{X}$ from the training data, and compute the gradient $d_t = \nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}_{t-1})$ of the sample. All steps below this are computed element-wise.
2. Update our first and second moment estimates, $\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) d_t$, and $\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) d_t^2$
3. Update our moment estimates to correct for bias, so $\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t}$ and $\hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1 - \beta_2^t}$
4. Update our schedule multiplier γ_t if we need to. This is usually kept constant at $\gamma_t = 1, \forall t$.
5. We finally update our parameters, such that $\boldsymbol{\theta}_t = \boldsymbol{\theta}_{t-1} - \gamma_t \left(\alpha \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon} + \lambda \boldsymbol{\theta}_{t-1} \right)$

Once we have reached our stopping criterion, we return our optimised parameters $\boldsymbol{\theta}_t$.

3. Proposed Solution

Our proposed model, AFM-MLP, combines two components that work together to capture feature interactions at different orders. The *AFM component* captures order-2 (pairwise) feature interactions using an attention mechanism that learns which feature pairs are most predictive. The *MLP component* captures higher-order feature interactions through its deep layered architecture.

3.1 Embedding Layer

The sparse feature vector $\mathbf{x} \in \{0, 1\}^n$ is high-dimensional and unsuitable for direct input into neural network components. The embedding layer transforms this sparse representation into a dense, low-dimensional representation.

For the j -th field with one-hot vector $\mathbf{x}_{\text{field}_j}$, the field embedding is:

$$\mathbf{v}_j = \mathbf{V}_j^\top \mathbf{x}_{\text{field}_j} \quad (4)$$

Since $\mathbf{x}_{\text{field}_j}$ is one-hot, this operation retrieves the row of $\mathbf{V}_j$ corresponding to the active category. The output of the embedding layer is the collection of m field embeddings $\mathcal{E} = \{\mathbf{v}_1, \dots, \mathbf{v}_m\}$. Note that while the lengths of different input field vectors $\mathbf{x}_{\text{field}_j}$ vary (since n_j differs across fields), their embeddings $\mathbf{v}_j$ are all of the same dimension k . This shared embedding serves as input to both the AFM and MLP components, allowing both to jointly influence how the embeddings are learned during backpropagation and reducing the number of parameters compared to architectures with separate embeddings.

3.2 AFM Component

3.2.1 FACTORISATION MACHINES (FM)

Before introducing AFM, we first review the standard Factorisation Machine. FM is comprised of linear terms and pairwise feature interactions:

$$\hat{y}_{FM}(\mathbf{x}) = w_0 + \sum_{i=1}^n w_i x_i + \sum_{i=1}^n \sum_{j=i+1}^n \hat{w}_{ij} x_i x_j \quad (5)$$

This interaction enables FM to estimate weights for feature pairs that rarely appear together in the training data.

3.2.2 PAIR-WISE INTERACTION LAYER

As stated in the introduction, a key limitation of FM's are that they assign all pairwise interactions equal importance, introducing noise. To address this, the pair-wise interaction layer computes the element-wise product between all pairs of embedding vectors:

$$f_{PI}(\mathcal{E}) = \{(\mathbf{v}_i \odot \mathbf{v}_j) x_i x_j\}_{(i,j) \in \mathcal{R}_{\mathbf{x}}} \quad (6)$$

Unlike FM which uses the inner product $\langle \mathbf{v}_i, \mathbf{v}_j \rangle$ to produce a scalar, AFM uses the element-wise product $\mathbf{v}_i \odot \mathbf{v}_j$ to produce a k -dimensional vector, retaining more information about the interaction for the attention network to process.

3.2.3 ATTENTION NETWORK

The attention network learns the importance of each feature interaction from data. Rather than learning attention scores a_{ij} directly as parameters (which would fail for unseen feature pairs), the network takes the interaction vector as input and outputs an importance score.

The unnormalised attention score is computed as:

$$a'_{ij} = \mathbf{h}^\top \text{ReLU}(\mathbf{W}(\mathbf{v}_i \odot \mathbf{v}_j) + \mathbf{b}) \quad (7)$$

The attention scores are then normalised via softmax:

$$a_{ij} = \frac{\exp(a'_{ij})}{\sum_{(l,r) \in \mathcal{R}_{\mathbf{x}}} \exp(a'_{lr})} \quad (8)$$

3.2.4 ATTENTION-BASED POOLING LAYER

The attention-based pooling layer computes the weighted sum of all interaction vectors:

$$f_{Att}(f_{PI}(\mathcal{E})) = \sum_{(i,j) \in \mathcal{R}_{\mathbf{x}}} a_{ij} (\mathbf{v}_i \odot \mathbf{v}_j) x_i x_j \quad (9)$$

This produces a single k -dimensional vector that summarises all pairwise feature interactions, with more predictive interactions contributing more to the representation.

3.2.5 AFM OUTPUT

The final AFM output is obtained by projecting the attention-pooled vector to a scalar and adding the linear terms:

$$y_{AFM} = w_0 + \sum_{i=1}^n w_i x_i + \mathbf{p}^\top \sum_{i=1}^n \sum_{j=i+1}^n a_{ij} (\mathbf{v}_i \odot \mathbf{v}_j) x_i x_j \quad (10)$$

3.3 Multilayer Perceptron (MLP) Component

3.3.1 INPUT TO THE MLP

The MLP receives the same embedding vectors used by the AFM component, concatenated into a single input vector $\mathbf{a}^{(0)}$.

3.3.2 HIDDEN LAYERS

The MLP consists of H hidden layers, where each layer applies a linear transformation followed by a non-linear activation function. For layer $\ell \in \{1, 2, \dots, H\}$, the forward propagation is defined as:

$$\mathbf{a}^{(\ell)} = \sigma \left(\mathbf{W}^{(\ell)} \mathbf{a}^{(\ell-1)} + \mathbf{b}^{(\ell)} \right) \quad (11)$$

The activation function introduces non-linearity, enabling the network to learn complex and higher-order feature interactions that would be impossible with purely linear transformations.

3.3.3 OUTPUT LAYER

After passing through H hidden layers, the final hidden representation $\mathbf{a}^{(H)}$ is projected to a scalar output:

$$y_{MLP} = \mathbf{W}^{(H+1)} \mathbf{a}^{(H)} + b^{(H+1)} \quad (12)$$

3.4 Combined Model Output

3.4.1 OUTPUT COMBINATION

The final prediction is obtained by summing the outputs from both components and applying the sigmoid function:

$$\hat{y} = \text{sigmoid}(y_{AFM} + y_{MLP}), \quad (13)$$

where $\text{sigmoid}(z) = \frac{1}{1+e^{-z}}$ maps the combined output to a probability $\hat{y} \in (0, 1)$.

The additive combination of y_{AFM} and y_{MLP} follows the design principle established by DeepFM (Guo et al., 2017), allowing both components to contribute independently to the prediction, where each component models different aspects of the feature interactions.

3.5 Time Complexity Analysis

We analyse the computational complexity of a single forward pass through the AFM-MLP model. Let $\bar{n}$ denote the number of non-zero features in the input.

3.5.1 AFM COMPONENT

The pairwise interaction layer computes $(\mathbf{v}_i \odot \mathbf{v}_j)x_i x_j$ for each pair of non-zero features, yielding $\binom{\bar{n}}{2} = O(\bar{n}^2)$ interaction vectors. Each element-wise product costs $O(k)$ operations.

For each interaction, the attention network computes $a'_{ij} = \mathbf{h}^T \text{ReLU}(\mathbf{W}(\mathbf{v}_i \odot \mathbf{v}_j)x_i x_j + \mathbf{b})$, where $\text{ReLU}(\cdot)$ is an activation function and $\mathbf{W} \in \mathbb{R}^{t \times k}$ with t being the attention factor. The matrix-vector multiplication $\mathbf{W}(\mathbf{v}_i \odot \mathbf{v}_j)$ costs $O(tk)$ per pair. The softmax normalisation over all pairs adds $O(\bar{n}^2)$, and the final weighted sum and projection by $\mathbf{p}^T$ cost $O(k\bar{n}^2)$ and $O(k)$ respectively.

Combining these terms, the AFM component has complexity $O(tk\bar{n}^2)$. As noted in Xiao et al. (2017), AFM has complexity quadratic in the number of non-zero features. Standard FM can be computed in $O(k\bar{n})$ using an algebraic reformulation (Rendle, 2010), but AFM cannot use this because the attention weights a_{ij} must be computed for each pair separately.

3.5.2 MLP COMPONENT

The forward pass through layer ℓ computes $\mathbf{W}^{(\ell)}\mathbf{a}^{(\ell-1)}$, costing $O(d_\ell d_{\ell-1})$ operations. Summing over all $H + 1$ layers gives

$$O\left(\sum_{\ell=1}^{H+1} d_{\ell-1} d_\ell\right). \quad (14)$$

For a network with constant hidden dimension $d_1 = \dots = d_H = d$, this simplifies to $O(mkd + Hd^2 + d)$.

3.5.3 OVERALL COMPLEXITY

The total time complexity of AFM-MLP for a single forward pass is

$$O\left(tk\bar{n}^2 + \sum_{\ell=1}^{H+1} d_{\ell-1} d_\ell\right). \quad (15)$$

When $\bar{n}$ is large, the $O(tk\bar{n}^2)$ term from AFM dominates. This increased cost over FM comes from computing attention weights for each pair individually.

4. Numerical Experiments

4.1 Experimental Setup

Data split and sampling. Due to computational constraints, we randomly sampled 10 million observations from the Avazu CTR dataset and split the sample into 50% training, 25% validation, and 25% test data. Hyperparameter tuning and model selection were performed using the validation set, with final performance reported on the held-out test set. An analysis of our 10 million samples shows that there are 17% positive samples (user click) and 83% negative samples (user didn't click), which capture the real-world case of imbalance data for this task.

Feature encoding and engineering. All inputs are categorical features some being high-cardinality. Each feature was integer-encoded and mapped to a learned embedding

vector, enabling scalable training without explicitly one-hot encodings. Embeddings were trained end-to-end jointly with all model parameters. We also added `hour.time` (hour of the day) and `day_of_week` (eg. Sunday) as we believe that these would be important features.

Training procedure and optimisation. All neural models (AFM, DeepFM, and AFM-MLP) were implemented in PyTorch and trained using mini-batch stochastic gradient updates with batch size $B = 1024$. We used the AdamW optimiser. To ensure consistent experimental behaviour, we fixed the random seed to 42 for dataset sampling, parameter initialisation, and training.

Early stopping and epoch budget. For all grid-search experiments, we trained models for at most $E_{\max} = 4$ epochs unless early stopping triggered earlier. We used early stopping based on validation AUC with patience $p = 1$: training was terminated once validation AUC failed to improve for one consecutive epoch. The checkpoint achieving the highest validation AUC was retained for evaluation.

Compute environment. Most experiments were conducted on an NVIDIA P100 GPU using Kaggle notebooks, with training time of approximately 5 minutes per epoch at batch size 1024. Due to time constraints, a subset of runs was also executed on local CPU machines, using the same preprocessing and training pipeline.

Evaluation metrics. We evaluate model performance using AUC (Area Under the ROC Curve) and log-loss (binary cross-entropy). AUC measures ranking quality and can be interpreted as the probability that a randomly chosen positive instance ($y = 1$) receives a higher predicted score than a randomly chosen negative instance ($y = 0$). Since CTR datasets are highly imbalanced, AUC is a widely used metric for assessing performance. Log-loss evaluates the quality of the predicted probabilities and penalises overconfident incorrect predictions, making it informative for assessing probability calibration. Higher AUC and lower log-loss indicate better performance.

4.2 Hyperparameter Tuning

Due to computational cost of training AFM-MLP models, hyperparameter tuning was performed sequentially over architecture, optimisation, and regularisation parameters. This strategy reduces the search budget relative to a full joint grid, but it implicitly assumes limited interaction between hyperparameter groups.

Grid	Hyperparameter	Values searched
Architecture	Embedding dimension (<code>embed_dims</code>)	$\{8, 16, 32\}$
	Attention dimension (<code>attention_dim</code>)	$\{8, 16, 32\}$
	MLP architecture (<code>mlp_dims</code>)	$\{(128, 128), (256, 256), (400, 400, 400)\}$
Optimisation	Learning rate (<code>lr</code>)	$\{5 \times 10^{-4}, 10^{-3}, 2 \times 10^{-3}\}$
	Weight decay (<code>weight_decay</code>)	$\{0, 10^{-6}, 10^{-5}\}$
Regularisation	Dropout rate (<code>drop_rate</code>)	$\{0.05, 0.10, 0.20, 0.30\}$
	Weight decay (<code>weight_decay</code>)	$\{0, 10^{-6}, 10^{-5}\}$

Table 1: Summary of hyperparameter grids explored for MLP-AFM.

Stage	Selected setting	Val AUC	Test AUC
Grid 1 (Architecture)	<code>embed=8, attn=8,</code> <code>mlp=(400,400,400)</code>	0.7435	0.7432
Grid 2 (Optimisation)	<code>lr=0.002, wd=0</code>	0.7435	0.7434
Grid 3 (Regularisation)	<code>drop=0.05, wd=0</code>	0.7436	0.7418
Final AFM-MLP (combined)	<code>embed=8, attn=8,</code> <code>mlp=(400,400,400),</code> <code>lr=0.002, drop=0.05,</code> <code>wd=0</code>	0.7437	0.7432

Table 2: Sequential tuning results and the final combined AFM-MLP model.

The best configurations found in each grid did not combine to produce a substantially stronger final model, with several intermediate settings achieving comparable if not stronger validation AUCs. This reflects strong interactions between model capacity, optimisation, and regularisation, which limit the effectiveness of sequential tuning. Nevertheless, the modest gap between validation and test AUC suggests limited overfitting during training.

4.3 Over-parameterised model with explicit regularisation

Weight decay provided little benefit in the tuned AFM-MLP, likely because early stopping and dropout already regularised the model. We therefore evaluated weight decay in an over-parameterised AFM-MLP to test whether explicit ℓ_2 regularisation becomes useful at higher capacity.

We fixed an over-parameterised AFM-MLP with `embed_dims = 32`, `attention_dim = 32`, `mlp_dims = (400, 400, 400)` and `drop_rate = 0`, and trained using AdamW with learning rate `lr = 0.002`, sweeping `weight_decay` $\in \{0, 10^{-6}, 10^{-5}, 10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}\}$.

Weight decay	Selected epoch	Val AUC	Test AUC	Val loss	Test loss
0	2	0.7422	0.7414	0.4046	0.4102
10^{-6}	3	0.7432	0.7398	0.4081	0.4161
10^{-5}	3	0.7425	0.7401	0.4092	0.4154
10^{-4}	3	0.7425	0.7401	0.4092	0.4154
10^{-3}	3	0.7429	0.7405	0.4088	0.4148
10^{-2}	2	0.7453	0.7448	0.4010	0.4063
10^{-1}	2	0.7556	0.7398	0.3939	0.4089

Table 3: Over-parameterised AFM-MLP with varying weight decay

Table 3 shows that weight decay becomes more effective in over-parameterised models. When model capacity is high, moderate weight decay improves generalisation by constraining embedding scale and limiting rapid overfitting. In particular, `weight_decay =  $10^{-2}$` achieves the best test AUC (0.7448) despite not maximising validation AUC. We observed similar performance for 10^{-5} and 10^{-4} indicating the minimal effect of weight decay for smaller values.

Stronger regularisation (`weight_decay` = 10^{-1}) maximises validation AUC but degrades test performance. This indicates that validation AUC is not perfectly predictive of generalisation and that overly strong weight decay can lead to unstable model selection and poorer out-of-sample performance. This is consistent with early stopping selecting underfitted yet strongly regularised solutions.

This also explains why weight decay had little effect in the sequentially tuned model: when capacity is moderate and dropout and early stopping are active, additional ℓ_2 regularisation provides limited benefit especially in cases where λ is smaller. Only in the high-capacity models does weight decay meaningfully improve generalisation.

Overall, these results confirm that regularisation effectiveness in AFM-MLP models depends strongly on model capacity and optimisation dynamics, and that hyperparameters must be tuned jointly rather than in isolation.

4.4 Performance Comparison

To showcase the performance of our model, we will compare them with classification models and models that have inspired our project. For DeepFM, we will set the embedding dimension to 8 and the structure of MLP as [400,400,400]. For AFM, we set the embedding dimension to 8 and the attention layer dimension to 8. For FM, we set the embedding dimension to 8. All of the models were trained at 6 epochs, as they start to overfit to the training sample after that. The AFM-MLP are trained using their optimal setting, as shown in the "Hyperparameter Tuning" section

Table 4: Test set performance of CTR prediction models on the Avazu dataset. Best results in bold.

Model	AUC $\uparrow$	Log-loss $\downarrow$
Logistic Regression	0.6396	0.4937
FM	0.7116	0.4610
AFM	0.7382	0.4057
DeepFM	0.7195	0.4402
AFM-MLP	0.7432	0.4036

From this table, we see that the AFM-MLP received the best performance in both AUC and Log-loss. The additional of attention mechanism resulted in big performance increase, compared to DeepFM. However, it must be noted that the performance of AFM-MLP is not a significant increase over AFM that does not contain the MLP part. This may suggest that high-order feature interactions is not as significant as that of low-order feature's.

5. Conclusion

The improvement of AFM-MLP over AFM is modest, indicating that attention-weighted second-order interactions account for most of the predictive gain in our setting. From a practical perspective, this suggests that AFM may be preferred when computational

resources are limited, while AFM-MLP offers a small accuracy improvement at increased training and inference cost.

A key limitation of our study is that the contribution of the MLP portion is difficult to interpret thus limiting our analysis of the effect of high order feature interactions. In addition, our hyperparameter search was constrained by compute, preventing fully joint tuning as well as multi-seed evaluations. Future work should therefore, try and quantify the effect of MLP alone doing experiments by itself as well as more extensive hyperparameter optimisation.

Our solution mainly focused on the second-order interactions, it would be interesting to consider up to order-3 feature interactions in our AFM model as a way of explicitly incorporating higher-order interactions into our model. However, we were limited by our computational constraint in our experiment, so it has not been explored.

Recent advancement in transformers and LLM-based models has also proposed new ways for CTR Prediction. Pure LLM-based (Yu et al., 2024) (Geng et al., 2024) and combination of LLM and traditional machine learning model (Zhang et al., 2025) have shown to beat the performances of DeepFM, due to their ability in capturing semantic information based on user’s previous interaction and product information with external knowledge pre-trained in the model. However, as this is not part of ST310’s course, we didn’t experiment with the following techniques but we believe that future of CTR prediction will lie in the following techniques.

References

- Heng-Tze Cheng, Levent Koc, Jeremiah Harmsen, Tal Shaked, Tushar Chandra, Hrishi Aradhye, Glen Anderson, Greg Corrado, Wei Chai, Mustafa Ispir, Rohan Anil, Zakaria Haque, Lichan Hong, Vihan Jain, Xiaobing Liu, and Hemal Shah. Wide deep learning for recommender systems, 2016. URL <https://arxiv.org/abs/1606.07792>.
- Binzong Geng, Zhaoxin Huan, Xiaolu Zhang, Yong He, Liang Zhang, Fajie Yuan, Jun Zhou, and Linjian Mo. Breaking the length barrier: Llm-enhanced ctr prediction in long textual user behaviors, 2024. URL <https://arxiv.org/abs/2403.19347>.
- Huifeng Guo, Ruiming Tang, Yunming Ye, Zhenguo Li, and Xiuqiang He. Deepfm: A factorization-machine based neural network for ctr prediction, 2017. URL <https://arxiv.org/abs/1703.04247>.
- Yuchin Juan, Damien Lefortier, and Olivier Chapelle. Field-aware factorization machines in a real-world online advertising system, 2017. URL <https://arxiv.org/abs/1701.04099>.
- Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization, 2019. URL <https://arxiv.org/abs/1711.05101>.
- Steffen Rendle. Factorization machines. In *IEEE International Conference on Data Mining (ICDM)*, pages 995–1000. IEEE, 2010.

- Jun Xiao, Hao Ye, Xiangnan He, Hanwang Zhang, Fei Wu, and Tat-Seng Chua. Attentional factorization machines: Learning the weight of feature interactions via attention networks, 2017. URL <https://arxiv.org/abs/1708.04617>.
- Yanwu Yang and Panyu Zhai. Click-through rate prediction in online advertising: A literature review. *Information Processing and Management*, 59(2):102853, March 2022. ISSN 0306-4573. doi: 10.1016/j.ipm.2021.102853. URL <http://dx.doi.org/10.1016/j.ipm.2021.102853>.
- Xiaohan Yu, Li Zhang, and Chong Chen. Explainable ctr prediction via llm reasoning, 2024. URL <https://arxiv.org/abs/2412.02588>.
- Guoxiao Zhang, Yi Wei, Yadong Zhang, Huajian Feng, and Qiang Liu. Balancing efficiency and effectiveness: An llm-infused approach for optimized ctr prediction, 2025. URL <https://arxiv.org/abs/2412.06860>.
- Junlin Zhang, Tongwen Huang, and Zhiqi Zhang. Fat-deepffm: Field attentive deep field-aware factorization machine, 2019. URL <https://arxiv.org/abs/1905.06336>.